

Scaling Large Language Models

Ryan Lee

October 5, 2026

1 Overview

Even well-resourced companies such as Meta or OpenAI cannot afford to spend compute sweeping hyperparameters and architecture changes on trillion parameter models: instead researchers first conduct **scaling law studies** on smaller models to predict how performance scales with more compute. I'll describe how to set up scaling studies with a **Warmup-Stable-Decay (WSD)** scheduler and ensure hyperparameter transfer with a technique called μP . I also present my experiments on extending μP to non-tied embedding models and looped language models, where layers are repeated in model depth.

2 Scaling Laws

What is a Scaling Law? A scaling law is an equation that relates how a neural network's reducible error (validation or test loss) changes with resources (model or dataset size). Empirically, for neural networks, this equation has been found to follow a simple power law [HNA⁺17]: loss is equal to a constant times the resource raised to a constant coefficient. That is, if you plot loss and compute on a log-log scale, you get a linear relation between model loss (decreasing) and resource.

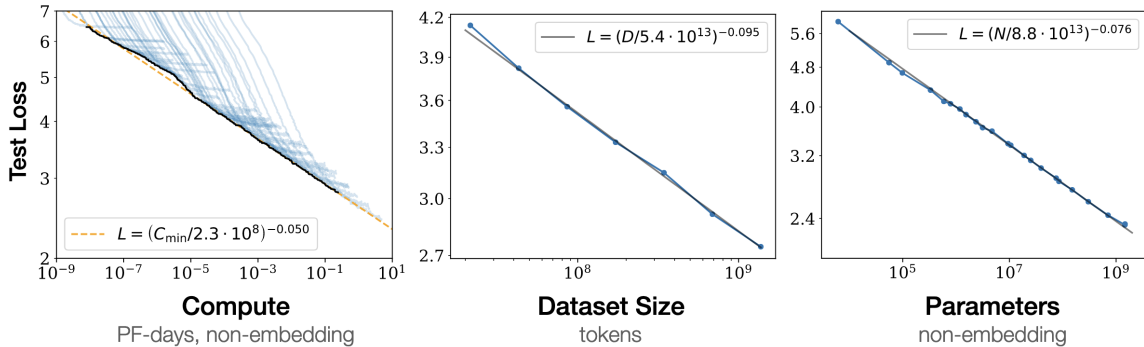


Figure 1: Empirical performance has a power-law relationship with each individual resource when not bottlenecked by the other two [KMH⁺20].

How can you use Scaling Laws? Scaling laws can be used to predict a much larger models performance with smaller scale models. On smaller scales, we can do many ablations on things like context length, model size, even dataset size, fit a power law equation, then see how they impact our efficiency on decreasing loss with increasing compute [KMH⁺20]. Authors have also used it to verify new architectures, like linear-time attention, GLU, and Mixture-of-Experts [TDA⁺23].

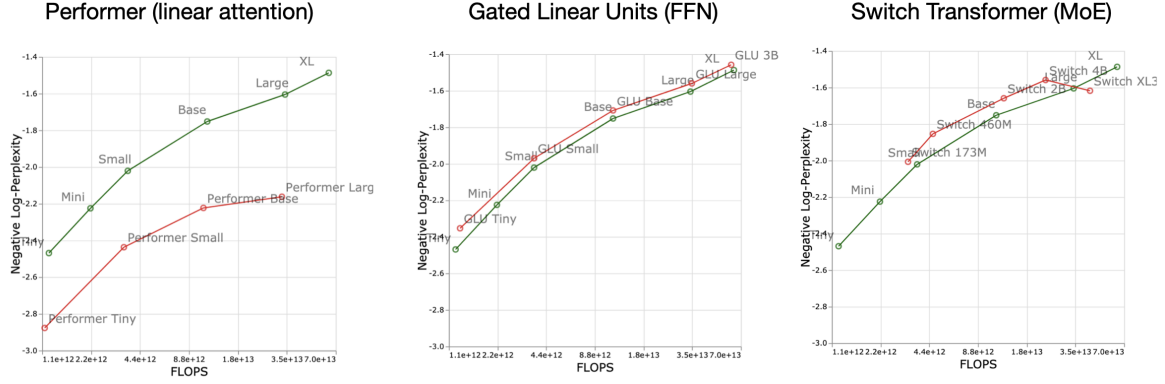


Figure 2: Note that it is Negative Log-Perplexity not log-loss, so higher curves are more favorable. We see that GLUs and Switch Transformers (Mixture-of-Experts) outperform the vanilla transformer, and these are the kinds of standard components now included in modern frontier models [TDA⁺23]

What does or doesn't follow Power Laws? To be as specific as possible, it is empirically found that for neural networks, (not generally all classifiers) cross-entropy loss follows a power equation with compute (**C**), as measured as floating-point-operations (FLOPs) on *non-embedding* parameters. It can be taken as a given then that number of tokens trained on (**D**) and model size in terms of number of non-embedding parameters (**N**) also follow a power law relation with loss, as they are directly related to compute. The relation between compute, model size and data size is estimated by the following:

$$C = 6ND \quad (1)$$

For non-embedding layers, the forward pass costs ~ 2 FLOPs per parameter (a multiply and an add). The backward pass costs ~ 4 FLOPs per parameter, as there are 2 for the gradient w.r.t. inputs and 2 for the gradient w.r.t. weights. Totaling, there are ~ 6 FLOPs per parameter total. Note that these are non-embedding layers, but it's justified to separate them, as embedding parameters are independent of sequence length and we also want to decouple vocabulary size from the compute budget.

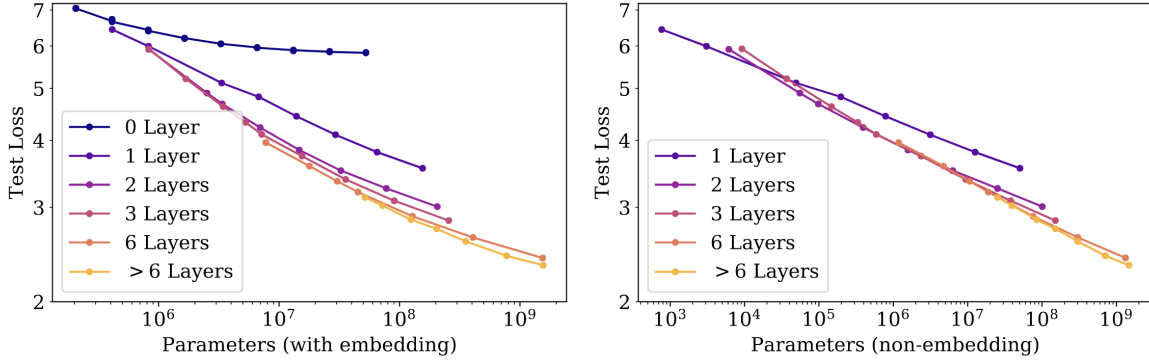


Figure 3: Left: When we include embedding parameters, performance appears to depend strongly on the number of layers in addition to the number of parameters. Right: When we exclude embedding parameters, the performance of models with different depths converge to a single trend. Only models with fewer than 2 layers or with extreme depth-to-width ratios deviate from the trend [KMH⁺20].

With a fixed compute budget, should N or D be larger? Because $C \approx 6ND$, choosing N determines D and vice versa. Hoffmann et al. estimate the compute-optimal trade-off using IsoFLOP profiles (their Approach 2): for each fixed compute budget, they train models of varying sizes and plot final loss against N , producing a U-shaped curve whose minimum identifies the optimal model size [HBM⁺22]. Fitting a power law through these minima gives $N_{\text{opt}} \propto C^a$ and $D_{\text{opt}} \propto C^b$ with $a \approx b \approx 0.50$, meaning model size and training tokens should be scaled in roughly equal proportions. This differs from Kaplan et al., who suggested scaling N much faster than D .

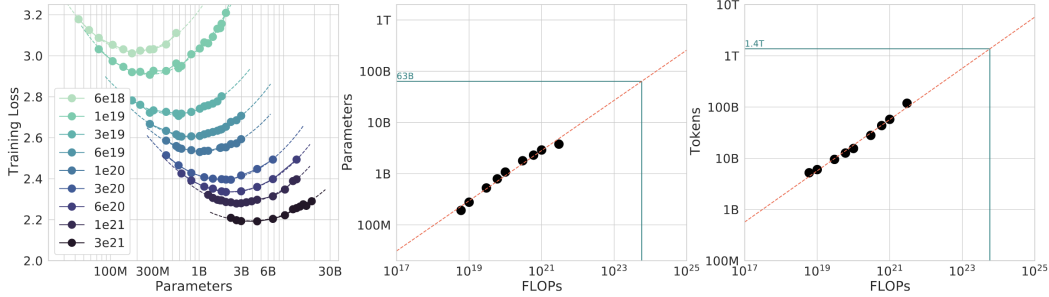


Figure 4: For each fixed compute budget, final loss is plotted against model size. The minimum of each curve identifies the compute-optimal model size for that budget.

3 Learning Rate Schedules for Efficient Scaling Studies

Cosine Decay. The standard approach to learning rate scheduling in large-scale pretraining has been the cosine schedule [LH17]: after a short warmup, the learning rate follows a cosine curve from peak to near-zero over a predetermined number of steps. Cosine decay has been the default in most scaling law studies [KMH⁺20, HBM⁺22], and it works well in practice. However, it has a significant drawback: the total number of training steps must be specified in advance. This means that every point on a scaling law—every (N, D) pair—requires a separate training run from scratch with its own cosine schedule. Fitting an isoFLOP curve thus requires training many models to completion, which is expensive.

Warmup-Stable-Decay (WSD). An alternative is the Warmup-Stable-Decay schedule [HTH⁺24]: a short warmup, followed by a long phase at constant (peak) learning rate, and finally a brief cooldown where the learning rate decays to zero. The loss curve under WSD looks non-traditional: During the stable phase, loss remains relatively flat and even higher than cosine, but during cool-down it drops sharply, often matching or beating the best cosine result.

Reusing a Single Training Run for IsoFLOP Analysis. The key practical advantage of WSD is that the stable phase can be extended indefinitely, and cooldowns can be branched off from any checkpoint along the way. For IsoFLOPs studies, this means you can use a single model run for all IsoFLOPs budgets which use that model.

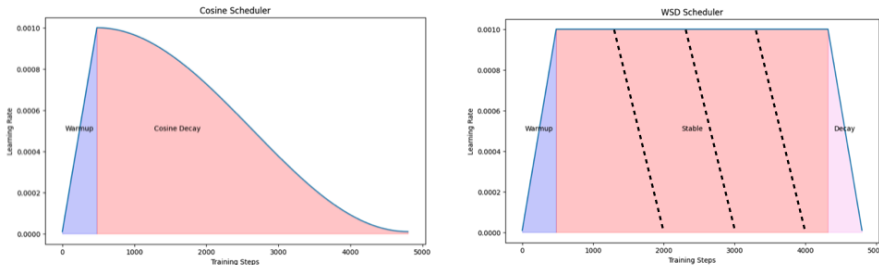


Figure 5: With WSD, you train mostly at one learning rate. WSD is empirically shown to be equivalent or better to cosine decay. WSD enables branching off from checkpoints for different isoFLOP budgets (see dotted lines on right figure). Notable LLMs that use WSD: MiniCPM, DeepSeekV3.

4 Scaling Learning Rate and Batch Sizes

Why don't Hyperparameters Don't Transfer Across Scales? There are two reasons, and maybe more. (1) matrix products multiply d_{model} elements, so our outputs get larger in magnitude when the hidden dimension increases, and (2) because we typically initialize based on input and output dimension, our initial weights also start larger for larger models. Specifically, we usually randomly initialize model weights to mean of zero and standard deviation $\sigma = \sqrt{2/(d_{in} + d_{out})}$. Our two main approaches to resolving this is to (1) scale down our gradient updates of weights of larger models and (2) scale down our initial standard deviation for larger models.

Maximal Update Parameterization ($\mu\mathbf{P}$). $\mu\mathbf{P}$ [YHB⁺22] is a reparameterization technique that scales initialization variance and per-layer learning rates as a function of model width, such that the magnitude of activations and weight updates remains constant regardless of scale. Concretely, for an Adam-based optimizer, $\mu\mathbf{P}$ prescribes the following. Define the width multiplier as:

$$m_d = \frac{d_{in}}{d_{in,base}} \quad (2)$$

where d_{in} is the fan-in of the target model and $d_{in,base}$ is the fan-in of the base (proxy) model. Then hidden-layer weights are initialized from:

$$W \sim \mathcal{N}\left(0, \frac{\sigma^2}{m_d}\right) \quad (3)$$

and the per-layer learning rate for hidden weights is scaled as:

$$\eta_{hidden} = \frac{\eta_{base}}{m_d} \quad (4)$$

Embedding and output layers follow slightly different rules, and I used my best judgment to apply the technique to a transformer that did not weight-tie the embedding and unembedding weights. The full set of scaling rules is summarized in Table 1, including a column for our implementation which uses untied embeddings (separate $\mathbf{W}_{out}$). I found that the implementation became a lot simpler: I remove the additional embedding forward-pass multipliers and output scaling in the original paper, but follow the prescribed initialization and learning rate scaling for everything else.

	SP	$\mu\mathbf{P}$	$\mu\mathbf{P}$ (Mine)
Embedding Init. Var.	σ_{base}^2	σ_{base}^2	σ_{base}^2
Embedding LR	η_{base}	η_{base}	η_{base}
Embedding Fwd.	$\mathbf{x}\mathbf{W}_{emb}$	$\alpha_{input} \cdot \mathbf{x}\mathbf{W}_{emb}$	$\mathbf{x}\mathbf{W}_{emb}$
Hidden Init. Var.	σ_{base}^2	σ_{base}^2/m_d	σ_{base}^2/m_d
Hidden LR (Adam)	η_{base}	η_{base}/m_d	η_{base}/m_d
Output Logit Fwd.	$\mathbf{x}\mathbf{W}_{emb}^\top$	$\alpha_{output} \cdot \mathbf{x}\mathbf{W}_{emb}^\top/m_d$	$\mathbf{x}\mathbf{W}_{out}$
Attention Logits	$QK^\top/\sqrt{d_{head}}$	$QK^\top/d_{head}$	$QK^\top/d_{head}$

Table 1: Comparison of Standard Parameterization (SP), $\mu\mathbf{P}$, and our variant with untied embeddings and no forward-pass multipliers. $m_d = d_{in}/d_{in,base}$ is the width multiplier.

Hyperparameter Transfer. The workflow is: train a small proxy model matching the target depth (number of transformer layers), and sweep to find the best settings for learning rate and batch size. Then apply we apply the optimal settings for the small model directly to the full-scale model. Some success stories: Yang et al. showed that a 200-sample random hyperparameter search on a 40M parameter model could be transferred to GPT-3 6.7B, achieving performance comparable to GPT-3 13B. Similarly, Dey et al. performed training recipe ablations at 111M parameters and transferred the results to a 3B model that matched contemporary 7B models while using 3.3 $\times$ fewer FLOPs [DGZ⁺23].

What are the limitations of $\mu\mathbf{P}$? One of the main limitations with this hyperparameter transfer technique are that depth and context window needs to remain fixed between small and large models. Additionally weight decay and dropout may not transfer and must be tuned separately¹. Some have found though, that $\mu\mathbf{P}$ transfer is not sensitive to batch size [Lin25].

¹I'm investigating this claim, not many ablations out there

Coordinate Check. The first diagnostic for validating a μP implementation is the **coordinate check** [YHB+22]. Take a small model (e.g., 2-layer GPT-2) and train it for a handful of steps (typically ~ 10) at several different widths (e.g., $d_{\text{model}} \in \{128, 256, 512, 1024, 2048\}$), averaged over multiple seeds. At each step, record the mean absolute value of activations at each layer. Under SP, this value grows proportionally with width: wider models have larger activations, which is precisely the source of hyperparameter shift and instability. Under a correct μP implementation, the activation magnitude should be *flat* across widths at every training step.

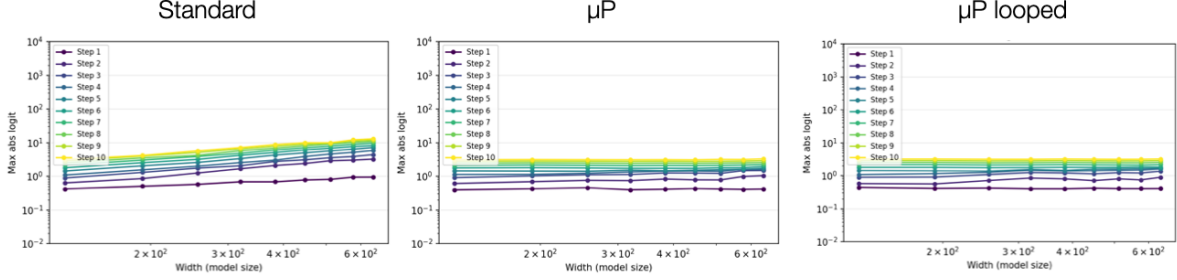


Figure 6: Coordinate check for my μP implementation. Mean absolute activation value of the logits is plotted across model widths over the first 10 training steps. Activation magnitudes remain flat across widths, confirming that our parameterization correctly controls the scale of activations independent of model size. I also tested with repeated or looped layers, and standard μP parameterization still works.

$\mu\text{Transfer}$ Test. The second and more definitive diagnostic is the **$\mu\text{Transfer}$ test**: a learning rate sweep across multiple model widths. For each width, train the model for a fixed number of steps and record the final loss as a function of learning rate. Under SP, the optimal learning rate shifts left (smaller) as the model gets wider. Under μP , the optimal learning rate should remain *the same* across all widths. This is the ultimate integration test: if the $\mu\text{Transfer}$ test passes, you can be confident that hyperparameters tuned on a small proxy model will transfer to your target scale.

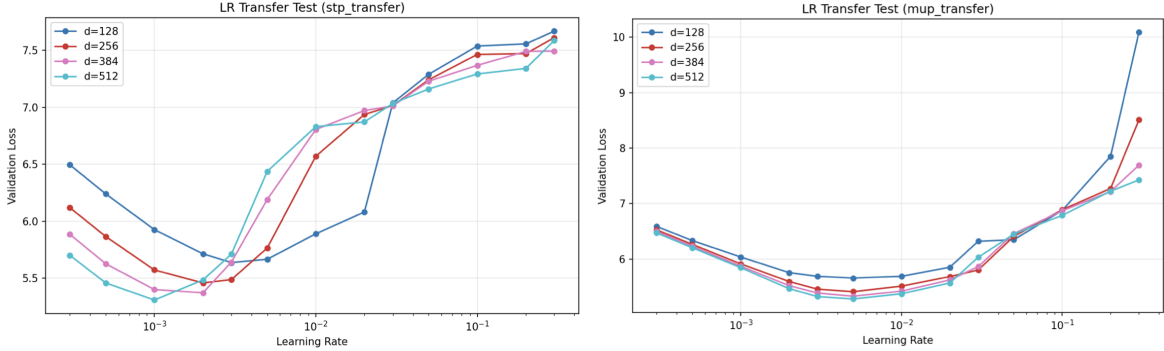


Figure 7: Final validation loss is plotted as a function of learning rate for several model widths. Shown is the transfer test for standard parameterization vs. μP , where μP has same ideal LR across size.

How do we apply μP to Mixture-of-Experts (MoE)? Unfortunately, the coordinate check, run over 10 steps, is not well suited for checking whether MoE outputs remain constant with respect to model width. This is in part due to the fact that the experts are sparsely activated: our weight scaling via scaled gradient updates don't happen in a consistent way (Recall that we have two auxiliary load balancing losses that ensure different experts are used and that only a few are activated). One paper suggests μP parameterizing the expert weights as hidden weights, and scaling only the output of the router (like output scaling done after the tied un-embedding weight is applied) [MCB+25].

References

- [DGZ⁺23] Nolan Dey, Gurpreet Gosal, Zhiming, Chen, Hemant Khachane, William Marshall, Ribhu Pathria, Marvin Tom, and Joel Hestness. Cerebras-GPT: Open Compute-Optimal Language Models Trained on the Cerebras Wafer-Scale Cluster, April 2023. arXiv:2304.03208 [cs].
- [HBM⁺22] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training Compute-Optimal Large Language Models, March 2022. arXiv:2203.15556 [cs].
- [HNA⁺17] Joel Hestness, Sharan Narang, Newsha Ardalani, Gregory Diamos, Heewoo Jun, Hassan Kianinejad, Md Mostofa Ali Patwary, Yang Yang, and Yanqi Zhou. Deep Learning Scaling is Predictable, Empirically, December 2017. arXiv:1712.00409 [cs].
- [HTH⁺24] Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu Cui, Xiang Long, Zhi Zheng, Yewei Fang, Yuxiang Huang, Weilin Zhao, Xinrong Zhang, Zheng Leng Thai, Kaihuo Zhang, Chongyi Wang, Yuan Yao, Chenyang Zhao, Jie Zhou, Jie Cai, Zhongwu Zhai, Ning Ding, Chao Jia, Guoyang Zeng, Dahai Li, Zhiyuan Liu, and Maosong Sun. MiniCPM: Unveiling the Potential of Small Language Models with Scalable Training Strategies, June 2024. arXiv:2404.06395 [cs].
- [KMH⁺20] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling Laws for Neural Language Models, January 2020. arXiv:2001.08361 [cs].
- [LH17] Ilya Loshchilov and Frank Hutter. Sgdr: Stochastic gradient descent with warm restarts, 2017.
- [Lin25] Lucas Lingle. An Empirical Study of $\text{\$}\text{\$}$ P Learning Rate Transfer, February 2025. arXiv:2404.05728 [cs].
- [MCB⁺25] Jan Małaśnicki, Kamil Ciebia, Mateusz Boruń, Maciej Pióro, Jan Ludziejewski, Maciej Stefaniak, Michał Krutul, Sebastian Jaszczur, Marek Cygan, Kamil Adamczewski, and Jakub Krajewski. μ -parametrization for mixture of experts, 2025.
- [TDA⁺23] Yi Tay, Mostafa Dehghani, Samira Abnar, Hyung Chung, William Fedus, Jinfeng Rao, Sharan Narang, Vinh Tran, Dani Yogatama, and Donald Metzler. Scaling Laws vs Model Architectures: How does Inductive Bias Influence Scaling? In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 12342–12364, Singapore, December 2023. Association for Computational Linguistics.
- [YHB⁺22] Greg Yang, Edward J. Hu, Igor Babuschkin, Szymon Sidor, Xiaodong Liu, David Farhi, Nick Ryder, Jakub Pachocki, Weizhu Chen, and Jianfeng Gao. Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer, March 2022. arXiv:2203.03466 [cs].